A BM REFERENCE

bm game API, volume 1

Drawing, input and system: what a Lua cartridge can call, with a first game to follow.

Contents

How a cartridge works 4

The functions your game defines 4

Screen sizes 4

Colours and coordinates 5

Limits 5

Leaving a game 5

The time budget 5

Your first game 6

Try it, step by step 6

Other ways to try it 6

Cover 7

Drawing 8

Screen and shapes 8

Sprites 10

The tile map 11

Input 14

Buttons 14

Keys chosen by the player 15

Mouse and pointer 16

More players 18

Time and system 19

`stat(n)` 20

Heavy frames: `frameskip` 21

The dev kit and the profiler 21

The last session's report 22

Examples 22

Saves 24

Good to know 24

A record 24

Several games in progress 24

Documents 26

Good to know 26

Screenshots 27

For the player 27

For an app 27

Names and format 27

Try it, step by step 28

Network (UDP) 29

Open points 30

README

What this is. Volume 1 of the bm game API: how a cartridge works, a first game step by step, then drawing, input, time and system, saves, documents, screenshots and the network.

How to use it. Do the first game in order, then keep the tables beside you. Anything in a bordered box with a shadow is meant to be copied as it is.

What is missing. Every gap is marked with a highlighted TO FILL tag; nothing is invented. Sound, 3D, bmlib, bmnet and the tools are in the other volumes.

How a cartridge works

A `.bm` cartridge is a game for bm written in **Lua 5.4**. The kernel draws in C (640×360, 16-bit RGB565 colour, 60 frames a second); Lua only does the logic. Complete examples are in `carts/` : `pong/` , `snake/` , `shooter/` (code only) and `demo/` (with a PNG sprite sheet and a CSV map).

The functions your game defines

Function	When
`_init()`	once, after loading
`_update()`	every frame (60 Hz), before `_draw`
`_draw()`	every frame, after `_update`
`_exit()`	(optional) Ctrl+Esc, PS or Start+Select: `true` closes now; `false` stays (the cartridge asks, for example "unsaved changes: Ctrl+Esc again leaves", and closes later with `quit()` or at the second Ctrl+Esc)
`_leave()`	(optional) in a network match (`online(true)`), when the player confirms leaving: the game tells the server (or the host), then the cartridge closes without being suspended

Put the logic and the input in `_update` , and only the drawing in `_draw` . The screen is **not** cleared by itself: `_draw` usually starts with `cls()` .

Screen sizes

`SCREEN_W` and `SCREEN_H` are globals that hold the size of the screen.

Cartridge made with	`SCREEN_W` × `SCREEN_H`
default	640 × 360
`--res 480x270`	480 × 270
`--res 320x180`	320 × 180
`--res 256x256`	256 × 256
`--res 360x360`	360 × 360 (the `.b16` 's screen)

The cartridge can change its resolution while it runs with `screen(w, h)` (see Drawing): from the next frame the new sizes hold.

Colours and coordinates

- Colours are integers `0xRRGGBB` (for example `0xFF8000`, orange) or `rgb(r, g, b)` with values 0–255. The screen turns them into RGB565 (5 bits red, 6 green, 5 blue).
- Coordinates are in pixels, (0,0) at the top left; `w` and `h` are width and height.

Limits

- An error or an endless loop (over **20 million Lua instructions** in one call of `_init`, `_update` or `_draw`; the counter restarts at each call) stops the cartridge without blocking the console. The error, with its line, appears on the console.
- Sandbox: no `io`, `os`, `load`, `dofile`. `require` loads only the libraries built into the kernel: `"assist"`, `"bm3d"`, `"bmlib"`, `"bmnet"`, `"predict"`, `"words"`, `"padtype"`, `"riff"` and `"bmui"`. You have `string`, `table`, `math`, `utf8` and `coroutine`.

Leaving a game

- Ctrl+Esc (keyboard), PS or Start+Select (gamepad) go back to bm's menu leaving the game **suspended**: it stays in memory, and A on its cover resumes it where it was. If the cartridge has `_exit()`, they ask it first.
- Esc alone, in a game, is Start (its menu).
- In a **network match** (`online(true)`) the game is not suspended: the system asks the player who leaves, only on their console, "Leave the match?". Yes calls `_leave()` and closes the game; back stays.
- From the serial line Ctrl+\ is Ctrl+Esc, and `q` closes without asking.
- The system's keys and the cartridge's are shown while **F12** is held.

The time budget

- 60 fps means **16.7 ms** per frame for `_update` + `_draw` + the copy to the screen. `stat(1)` tells you how much the cartridge uses.
- Drawing is in C: a `spr` or `rectfill` call costs a few microseconds, but every call from Lua has a fixed cost. On the Pi: about 1800 sprites of 16×16 called from Lua at 60 fps, about 4500 from C; about 1200 3D triangles.
- Avoid creating new tables in every frame if you can: it is less work for the garbage collector.
- Text is in 8×16 cells: for well aligned writing use x multiples of 8 and y multiples of 16.

Your first game

You will make a yellow ball you move with the arrows, on a dark blue screen. It uses only `cls`, `circfill`, `print` and `btn`.

Try it, step by step

1. On your computer, in the bm repository, create the file `carts/hello/main.lua` with this code:

```lua
local x, y = 300, 160

function _init() -- once, at start end

function _update() -- 60 times a second: logic if btn(0) then x = x - 3 end -- left if btn(1) then x = x + 3 end -- right if btn(2) then y = y - 3 end -- up if btn(3) then y = y + 3 end -- down end

function _draw() -- 60 times a second: drawing cls(0x102040) -- dark blue background circfill(x, y, 20, 0xFFD050) -- a yellow ball print("hello from bm!", 8, 8, 0xFFFFFF) end
```

2. Pack it into a cartridge. You should get a file `hello.bm` in the current folder.

```
sh python3 scripts/mkbm.py -o hello.bm --lua carts/hello/main.lua --title "Hello" --author "me"
```

3. Put it on the console. Copy `hello.bm` into the `carts/` folder of the SD card and power on. If the console is already on, press `R` in the menu: the card is read again.
4. In the menu, find **Hello** by "me" and start it. You should see a dark blue screen, a yellow ball and the white text `hello from bm!` at the top left.
5. Press the arrows (or WASD): the ball moves 3 pixels each frame.
6. Press Ctrl+Esc (keyboard) or Start+Select (gamepad) to go back to the menu. The game stays suspended; A on its cover resumes it.

If you see an error instead of the ball, the Lua code went wrong: the game stops and the message with the line number appears on the console. Fix that line and pack again.

[TO FILL: shots/api-core-hello.png]

TRY IT

Make the ball change colour when you press A. In `_update`, add `if btnp(4) then c = rgb(math.random(255), math.random(255), math.random(255)) end`, declare `local c = 0xFFD050` at the top, and draw the ball with `circfill(x, y, 20, c)`.

Other ways to try it

The repository's build can add your game by itself, and you can also try it without a Pi.

- In the build: put the game in `carts/<name>/main.lua`, add `<name>` to `GAMES` in the `Makefile` and a line `title_<name> := Title`. `make` creates `build/carts/<name>.bm`.
- In QEMU, without a Pi:

COPY AND PASTE

```
qemu-system-arm -M raspi0 -bios build/kernel.img -serial tcp:127.0.0.1:4444,server=on,wait=off -serial n
python3 tools/bm_load.py tcp:127.0.0.1:4444 --cart hello.bm
```

- From the serial line (a Pi with a USB-serial adapter):

COPY AND PASTE
```
python3 tools/bm_load.py /dev/ttyUSB0 --cart hello.bm
```

Cover

`--cover cover.png` is optional: a PNG of any size that the menu prints on the game's card, an 88×88 square. A square picture is scaled down; others stay whole, as wide as the square, over a blurred and darker copy of themselves that fills the rest. In the build a file `carts/hello/cover.png` is enough. Without a cover the menu prints the title.

Drawing

Everything on the screen: shapes, text, sprites and the tile map.

Screen and shapes

Function	What it does
`rgb(r, g, b)`	the colour integer `0xRRGGBB` of three numbers 0-255 (the API's colours are 24-bit integers: `0xFFFFFF` is white, a small number such as 7 is nearly black)
`cls([c])`	fills the screen (black if `c` is missing). With the 3D on the GPU the ARM does not fill it: the GPU's job clears the page to that colour (free even at 1080p); if 2D comes before the 3D the ARM fills it as always
`screen(w, h)`	sets the cartridge's resolution from the next frame: `true`, or `false` if it is not one of the sizes below
`screen()`	returns the width and height now
`screen(i)`	returns the i-th size of the list (from 1), or `nil`
`pset(x, y, c)`	writes a pixel
`pget(x, y)`	reads a pixel: `0xRRGGBB`, or `nil` off screen
`line(x0, y0, x1, y1, c)`	a line
`rect(x, y, w, h, c)`	an empty rectangle
`rectfill(x, y, w, h, c)`	a filled rectangle
`circ(x, y, r, c)`	an empty circle
`circfill(x, y, r, c)`	a filled circle
`tri(x0, y0, x1, y1, x2, y2, c, [c1, c2])`	a filled triangle; with three colours (one per corner) the colour shades from one corner to the other (Gouraud, dithered)
`print(text, x, y, [c, scale])`	text with the 8×16 font (white if `c` is missing), `scale` times bigger (1–8: 2 = 16×32 characters); returns the x after the last character
`font([name])`	sets the font of `print` from now on: `"8x16"` (the normal one), `"8x14"` or `"6x12"` (106 columns by 30 rows at 640×360: for tools with a lot of text); returns the width and height of a character of the current font
`prompt(name, x, y, [small, scale, player, menu])`	draws a key as a coloured chip, with its top left corner at (x, y); returns the x after the chip (details below)
`prompt(name, [small, scale, player])`	without coordinates it does not draw: it returns the width and height
`camera([x, y])`	moves all drawing by (–x, –y); without arguments it resets it

Function	What it does
`clip([x, y, w, h])`	limits drawing to the rectangle; without arguments the whole screen

Sizes for `screen(w, h)`

The list is 16:9: 320×180, 384×216, 480×270, 640×360, 960×540, 1280×720, 1920×1080. On a 1080p TV all are at whole pixels but 1280×720.

- `SCREEN_W` and `SCREEN_H` change when it is done; the z-buffer, the 3D camera and the light buffers follow, the font and `camera()` stay, `clip()` goes back to the whole screen.
- If the console cannot give the size, the one before stays.
- On the **RGB30** (a square panel) the list is the `.b16` 's two sizes instead: 360×360 (shown twice as big) and 720×720, both at whole pixels.
- A square cartridge (256×256, 360×360) changes as the others: on the Pi it then fills the TV, and the GPU can draw its 3D there (in the box in the middle the ARM draws it).
- The ARM's rasteriser pays for every pixel: on the Pi above 640×360 the GPU is needed.

Button and key chips: `prompt`

`prompt` draws a key like a coloured chip (the apps' set, `src/kernel/prompts.c`). It is 16 pixels high beside the 8×16 text, 12 with `small` (by itself when the font is `"6x12"`), `scale` times bigger (1–8, as `print`).

`name`	What is drawn
upper case: `"A"`, `"B"`, `"X"`, `"Y"`, `"START"`, `"SELECT"`, `"L1"` … `"R3"`, `"UP"`, `"UPDOWN"`, `"LEFTRIGHT"`, `"DPAD"`, `"LSTICK"`, `"PS"`, `"TOUCHPAD"`	the pad's buttons, drawn as on the pad used last: a DS4 (cross, circle, square, triangle, OPTIONS, SHARE) until another one is used, which has them with letters
`"RGB30_A"`, `"RGB30_B"`, `"RGB30_X"`, `"RGB30_Y"`	the RGB30's own buttons, on any console
lower case: `"mouse"`, `"mouse_l"`, `"mouse_r"`	the mouse; `"mouse_l"` and `"mouse_r"` with the left or the right button in red
lower case: `"enter"`, `"esc"`, `"tab"`, `"space"`, `"up"`, `"f1"` … or a character (`"s"`, `"1"`, `"+"`)	the keyboard's keys, with the names of `keyp()`
`"ok"`, `"back"` and the actions of `keymap()`	their button (on the RGB30 `"ok"` is B and `"back"` A, swapped by `confirm=a`: the chip is the button really pressed)

- On the **RGB30** `"A"`, `"B"`, `"X"`, `"Y"` are the console's own buttons (Nintendo layout: X top, A right, B bottom, Y left), a dark button with the letter in its colour: A green, B blue, X red, Y yellow (until a Bluetooth pad is used).
- With `player` (1–4, after `scale`: `prompt(name, x, y, small, scale, player)`) the button is drawn as that player's controller shows it: the DS4's symbol, a pad's letter or the keyboard key that presses it (`"space"` for A).
- With `menu` true (the seventh argument) you get the menu's set instead: white keys, raised on a lip, always 16 pixels high (`small` is ignored).

```
local x = prompt("ok", 16, 200)           -- the chip; returns the x after it
print("start", x + 4, 200, 0xFFFFFF)
prompt("RGB30_X", 16, 220)                -- the RGB30's red X, on any console
```

[TO FILL: shots/api-core-prompt.png]

Sprites

The **sprite sheet** is a picture divided into 8×8 cells: cell `n` is at column `n % (width/8)` and row `n // (width/8)`.

- It comes from `--sheet sheet.png` (RGB or RGBA PNG; alpha under 128 = transparent). Without a PNG it is a transparent 256×256 sheet to draw with `sset` (32 cells per row: see `carts/shooter`).
- The sheet can be up to 4096×4096 pixels.
- A big sheet with at most 256 colours goes into the cartridge with `--sheet sheet.png --sheet8`: a palette and repeated runs (RLE) instead of 4 bytes per pixel, decoded at loading (Titan Clash: 2048×3376 pixels in 1.7 MB).

Function	What it does
`spr(n, x, y, [w, h, flip_x, flip_y])`	draws cell `n` (w×h cells, default 1×1), mirrored too
`sspr(sx, sy, sw, sh, dx, dy, [flip_x, flip_y, zoom])`	copies any rectangle of the sheet; with `zoom` (default 1) it draws it bigger (`2`, `3` ...) or smaller (`0.5`), pixel by pixel: it covers `sw * zoom` × `sh * zoom` pixels
`sget(x, y)`	reads a pixel of the sheet (`nil` = transparent)
`sset(x, y, [c])`	writes a pixel of the sheet (`nil` = transparent)

```
spr(0, x, y)                  -- cell 0 (8x8)
spr(2, x, y, 2, 2)            -- 2x2 cells from cell 2: a 16x16 sprite
spr(2, x, y, 2, 2, true)      -- mirrored horizontally
sspr(sx, sy, w, h, x, y)      -- any rectangle of the sheet, in pixels
```

Named zones

The SPRITES section gives names to rectangles of the sheet, with their frames (the boxes of the same size to the right of the first) and the animation's speed. The code no longer needs the coordinates.

Function	What it does
`zspr(name, x, y, [frame, flip_x, flip_y, zoom])`	draws a **named zone** of the sheet: `frame` from 1 to the number of frames (beyond it starts again), or `nil`: the one its fps give at `time()`, so the animation runs by itself. Returns the frame drawn. A zone that is not there is an error
`zone(name)`	returns `x, y, w, h, frames, fps` of the zone (its first frame, in sheet pixels), or `nil`
`zones()`	the names of the zones, in order (`{}` if there are none)
`zboxes(name, [frame, kind])`	the **hitboxes and hurtboxes** of the frames of a zone (see below)

```
zspr("coin", x, y)                    -- the coin's animation, it runs by itself
zspr("hero_run", x, y, f, to_the_left)
local _, _, w, h = zone("hero_run")    -- its size, for collisions
```

Hitboxes and hurtboxes of the frames

Under a zone, in the zones file, you write its boxes: `hurt` where the character can be hit, `hit` where its blow hurts, `body` the room it takes (so that bodies do not pass through each other), with the frame (from 1, or `*` for every frame) and the rectangle from the frame's corner.

`zboxes` returns a list of `{x, y, w, h, kind, frame}` :

- `x` and `y` are from the frame's top left corner (the box may stick out).
- `kind` is `"hurt"` (where it can be hit), `"hit"` (where it hurts), `"body"` (the room the body takes) or a number of the game (3–255).
- `frame` is 0 for a box of every frame.
- With `frame` (from 1; past the last it starts again) you get its boxes and those of every frame; without, all the zone's.
- `kind` (a name or a number) keeps one kind.
- You get `{}` if there are none; a zone that is not there is an error.

The game puts the boxes in the world (mirrored too) with bmlib's `lib.hits` (see the bmlib volume).

```
# name  x  y  w  h  frames fps
punch   0  64 32 32 4 12
  hurt  *  8  2 16 30        # the body, in every frame
  hit   3  24 10 10  6       # the fist, only in frame 3
```

```
for _, b in ipairs(zboxes("punch", f, "hit")) do
  rect(x + b.x, y + b.y, b.w, b.h, 0xFF4040)     -- to see them while making the game
end
```

The tile map

The map comes from `--map map.csv` : one row of comma-separated numbers per row of the map, each number a cell of the sheet (0 = empty). Without a map it is 256×256 empty cells, one layer.

Function	What it does
`map(mx, my, [x, y, mw, mh, layer, mask])`	draws the map from cell (mx, my), mw×mh cells, at (x, y). `layer` : the number (from 1) or the name of the layer (default 1). `mask` : only the cells whose tile has at least one of those **flags** (`fget` ; 0 or missing: all)
`mget(mx, my, [layer])`	reads a cell of the map (0 = empty; outside the map it gives 0)
`mset(mx, my, n, [layer])`	writes a cell of the map
`msize([w, h])`	returns `w, h, layers` : the map's size in cells and how many layers it has; with `w` and `h` every layer gets that size, the cells that fit staying where they were

Function	What it does
`mlayers([list])`	returns the names of the layers, in order (the first is drawn at the back). With a list (1 to 8 entries) the map gets those layers: an entry is a name (the layer with that name, or a new empty one) or `{name, from}` (a copy of layer `from`, a number or a name): so they are added, moved, renamed and taken away
`fget(n, [f])`	the **flags** of tile (sheet cell) `n`: a byte, 8 flags; with `f` (0–7) that flag, `true` or `false`
`fset(n, f, on)`	turns flag `f` of tile `n` on or off
`fset(n, byte)`	with two arguments it writes all 8 flags at once
`mflags(x, y, [w, h, layer])`	the flags of the tiles the rectangle touches, **in pixels** (the map drawn at 0, 0; without `w` and `h` a point), put together (or); 0 = nothing. Cell 0 and the outside of the map have none. For collisions: `mflags(x, y + h, w, 1) & 1 ~= 0` (flag 0 under the feet)

```
camera(cam_x, cam_y)                                  -- scrolling
map(cam_x // 8, cam_y // 8, cam_x // 8 * 8, cam_y // 8 * 8, 81, 46)   -- only the part that shows
local t = mget(px // 8, py // 8)                      -- the cell under a point
mset(cx, cy, 5)                                       -- changing a cell (doors, objects taken)
```

Layers

The map has 1 to 8 **layers** of the same size, each with a name (the first is called `"main"` unless given another). They are drawn one at a time, in the order you like: for example the background, then the sprites, then the layer that goes **in front of** the character (tree tops, arches, roofs). Each layer is read and written by itself.

```
cls(0)
map(0, 0, 0, 0, 80, 45, "back")     -- the floor
spr(hero, x, y)
map(0, 0, 0, 0, 80, 45, "front")    -- the tree tops over the hero
mset(cx, cy, 0, "front")            -- each layer is read and written by itself
```

Flags: what each tile is

Every tile of the sheet has 8 flags (0–7) that say what it is to the game: wall, water, ladder, danger… Collisions look at the flags instead of the tiles' numbers, so you can add new tiles without touching the code. The convention of bmlib (every game can choose another):

Flag	Value	Meaning
0	1	solid: walls, floors
1	2	platform: you can pass through from below, stand on from above
2	4	ladder
3	8	water

Flag	Value	Meaning
4	16	hurts (spikes, lava)
5–7		free for the game

```
fset(12, 0, true)                              -- tile 12 is solid too
if fget(mget(cx, cy), 0) then ... end          -- the cell is solid
if mflags(px, py + 8, 8, 1) & 1 ~= 0 then      -- the character (8x8) stands on something
  on_ground = true
end
map(0, 0, 0, 0, 80, 45, 1, 4)                  -- only the ladders
```

Where sprites, maps and flags come from

With `mkbm.py` :

- `--map map.csv` is the first layer; every other `--map name=file.csv` is one more layer (up to 8, in the order given; a CSV smaller than the first is filled with empty cells).
- `--flags flags.csv` gives the flags: numbers 0–255 for cells 0, 1, 2… in order, or `n=flags` for cell `n` .
- `--sprites zones.txt` gives the zones, one per line: `name x y w h [frames [fps]]` . Under a zone, its boxes: `hurt` , `hit` or `body frame x y w h` , or `box kind frame x y w h` for a kind of the game. The names `hurt` , `hit` , `body` and `box` are not zones.
- In the repository's build the files `map_<name>.csv` (in order with `layers_<game> := name ...` in the `Makefile`), `flags.csv` and `sprites.txt` in the game's folder are enough.

```
# flags.csv: tile 1 is a wall (1), tiles 2 and 3 platforms (2), tile 9 a ladder (4)
1=1 2 2
9=4
```

On the console:

- On the SDK's map page `l` goes to the next layer, Shift+L adds one (also from the menu, *New map layer*), `o` shows that one alone, `c` the flags of the tiles.
- On the sprite page keys `0` – `7` toggle the cell's flags on and off.
- bm Pixel and `scripts/bmres.py` make zones (`bmres.py` carries the boxes with their zones).
- `cart_save` and `cart_load` carry layers, flags, zones and boxes with the cartridge.
- The format is in the LAYERS (12), FLAGS (13), SPRITES (11) and BOXES (14) sections of `src/bm/bm.h` . A kernel of before draws the first layer and ignores the rest.

Input

Buttons, the keyboard, the mouse and the players.

Buttons

`btn(i)` is true while the button is held, `btnp(i)` only in the frame it is pressed.

Function	What it does
`btn(i, [p])`	`true` while the button is held; without `p` from **any** controller, with `p` = 1–4 only from player `p`'s
`btnp(i, [p])`	`true` only in the frame it is pressed (the same `p`, also with a name)
`keymap(t)`	sets the game's **actions** on the buttons (see below)
`keymap()`	returns the table
`keymap(nil)`	takes the table away
`controller([p])`	returns what player `p` (1–4, the first if missing) plays with (see below)
`online([on, note])`	tells the system the match is played over the **network** (see below)
`players()`	how many players have a controller (at least 1) and, as the second value, which: bit `n` = player `n+1` (for example `3, 7` = players 1, 2 and 3)
`stick([p, n])`	player `p`'s left stick: `x, y` between –1 and 1 (x to the right, y down), with a dead zone; with the keyboard or a pad without a stick it is the cross (8 directions). With `n` = `1` the **right** stick (to aim in shooters; `0, 0` without a stick). Without `p`: the one pushed most

Instead of `i`, `btn` and `btnp` also accept a **name**:

- a button: `"a"`, `"b"`, `"x"`, `"y"`, `"left"`... `"down"`, `"start"`, `"select"`, `"l1"`, `"r1"`, `"l2"`, `"r2"`, `"l3"`, `"r3"`;
- `"ok"` and `"back"`, the system's yes and back: cross and circle on the DS4, A and B on an Xbox pad, Space and X on the keyboard; on the RGB30 as `confirm=` and `game_buttons=` say;
- an action of `keymap()`.

i	Button	Keyboard	Gamepad	Serial
0	left	← or A	cross / stick	`a` or ←
1	right	→ or D	cross / stick	`d` or →
2	up	↑ or W	cross / stick	`w` or ↑
3	down	↓ or S	cross / stick	`s` or ↓
4	**A**	space, Z, J	A / cross (DS4)	space, `j`
5	**B**	X, K	B / circle	`x`, `k`

i	Button	Keyboard	Gamepad	Serial
6	**X**	C, L	X / square	`c` , `l`
7	**Y**	V, I	Y / triangle	`v` , `i`
8	**Start**	Enter	Start / Options	Enter
9	**Select**	Tab	Select / Share	Tab

- Start+Select together (or the PS button) always close the cartridge: Start alone is free for the game's pause.
- A cartridge that never asks `btn(6)` / `btn(7)` (or `btnp`) gets X as A and Y as B: games with two buttons work with all four.

Actions: `keymap`

```
keymap({ jump = "a", fire = {"x", "r1"}, pause = "start", confirm = "ok" })
```

- Up to 32 actions, 4 buttons each. Then use `btn("jump", p)` , `btnp("fire")` and `prompt("jump", x, y)` .
- The game changes the keys by calling `keymap` again (its options menu; it keeps them with `save()`).
- An unknown button, or an action with a button's name, is an error.

What a player plays with: `controller`

`controller` returns `{kind, layout, bluetooth, ok, back}` and `color` :

- `kind` : `"keyboard"` , `"ds4"` , `"xbox"` , `"pad"` , `"builtin"` or `"none"` .
- `layout` : `"keyboard"` , `"ds4"` , `"xbox"` , `"nintendo"` or `"none"` . It says what the buttons are called: the DS4's symbols; letters with A at the bottom as Xbox; A on the right as the RGB30.
- `bluetooth` : a boolean.
- `ok` (`"a"` or `"b"`) and `back` (`"b"` or `"a"`): which of the game's buttons are the yes and the back.
- `color` : the player's colour, that of their pad's light, as an RGB integer: `0x3070FF` (player 1, blue), `0xFF3C28` (2, red), `0x28D848` (3, green), `0xFF38A8` (4, pink).

Network matches: `online`

- With `online(true)` , PS, Ctrl+Esc and Start+Select do not suspend the game (the others go on playing) but ask the player who leaves, only on their console, "Leave the match?" (they will leave the game and disconnect from the server), over the game that goes on.
- While the question is open the game sees neither buttons nor sticks nor keys. ok (cross on the DS4, Enter or Space on the keyboard) or PS again leave: `_leave()` and the cartridge closes. Back (circle, Esc) stays, and the buttons still held go back to the game only after being released.
- `note` is a line under the question (for example `"You are the host: the match ends for all."`).
- `online(false)` at the end of the match. `online()` returns whether it was online and whether the question is open (the player is away).

Keys chosen by the player

A cartridge that lets the player choose the keys (as nano8) reads the keyboard key by key and the controllers button by button.

Function	What it does
`rawkeys(on)`	with `true` the keyboards stop being controllers for `btn()` and `pad()`: they are read with `keydown()`. Ctrl+Esc still closes the cartridge
`keydown(u)`	`true` while the key with USB HID usage `u` is held (USB or Bluetooth): `0x04` … `0x1D` the letters A–Z, `0x1E` … `0x27` the digits, `0x28` Enter, `0x2C` space, `0x4F` … `0x52` the arrows (right, left, down, up), `0xE0` … `0xE7` Ctrl, Shift, Alt, GUI on the left and then on the right
`keys()`	the usages of the keys held now (`{0x1D, 0xE1}`): for "press a key"
`pad([p])`	the buttons player `p` (1–4) holds, as bits: 1 left, 2 right, 4 up, 8 down, 16 A, 32 B, 64 Start, 128 Select, 256 X, 512 Y, 1024 L1, 2048 R1, 4096 L2, 8192 R2, 16384 L3, 32768 R3 (the triggers and the pressed sticks: DS4 and Xbox 360; on generic pads buttons 7–8 and 11–12); without `p` everybody's. The serial keys count as the first player's controller (L1 and R1: `u` and `o` from the serial line, Q and E from the USB keyboard; Select: Tab from both)
`lastinput()`	what was pressed last: `"keyboard"`, `"ds4"` or `"pad"` (another controller); `nil` before any key. To show the right keys with `prompt()` (for example `"enter"` or `"A"`)

Mouse and pointer

A cartridge has the pointer only if it asks for it: without `mouse(true)` there is none (in bm's menu there always is). Only a USB or Bluetooth mouse moves it: a controller never shows it (its right stick is `stick(p, 1)`). The console may have it off for the whole system (`mouse=off` in `bm/config.txt`).

Function	What it does
`mouse(on, [arrow])`	`mouse(true)`: the cartridge wants the pointer, and the console draws its arrow over the frame (`mouse(true, false)`: no arrow, the cartridge draws its cursor); `mouse(false)` takes it away. Returns `false` if the console has the mouse off
`mouse()`	returns `x, y, buttons, wheel, visible`: the position in pixels of the cartridge's screen (without `camera`), the buttons held as bits (1 left, 2 right, 4 middle), the wheel's clicks in this frame (up positive) and `true` if the pointer shows (as soon as something moves it). `nil` if the cartridge has not asked for it or if nothing moves it (neither mouse nor right stick)
`mousep([i])`	`true` in the frame button `i` is pressed (0 left, the default; 1 right; 2 middle)

```
function _init() mouse(true) end
function _update()
  local x, y = mouse()
  if x and mousep() then tx, ty = x, y end   -- a click
end
```

bmui, the mouse of bm's tools

`require "bmui"` (`src/script/bmui.lua`). The editors (SDK, bm Code, bm Studio, bm Animator, bm Mesh, bm Pixel, Sound, the assistant's panel) use it, and a game or a tool of your own can too. Its rule: the mouse does what the keys do. Things are made clickable as they are drawn, and a click presses the key behind them or chooses what they show; the right button opens a context menu whose entries are keys.

Function	What it does
`U.update()`	call it first in `_update` (reads `mouse()` once a frame)
`U.begin()`	call it first in `_draw` (forgets the zones of the frame before)
`U.zone(x, y, w, h, kind, a, b)`	something clickable, as it is drawn (the last one drawn wins: a dialog over the page)
`U.at()`	the zone under the pointer
`U.click([b])`	the zone pressed this frame (0 left, 1 right, 2 middle)
`U.clicked(b)`	a button let go without a drag (the zone where it went down, or `true`)
`U.key_zone(x, y, w, h, keys)`	a chip (`"ctrl s"` , `{"shift", "l"}` , `"f5"`) that presses its key when clicked
`U.keys()`	presses the key of the chip clicked in this frame
`U.press(key)`	a key as if typed: the next `keyp()` gives it (bmui puts its own `keyp` first)
`U.menu(items, [x, y])`	the context menu at the pointer: `{ {"Copy", "^c"}, "-", {"Run", function() ... end} }` (the key shown on the right, "Ctrl+C"); rows on the 8x16 grid
`U.menu_update()`	in `_update` (true: it took the input)
`U.menu_draw()`	last in `_draw`
`U.x` , `U.y` , `U.on` , `U.moved` , `U.wheel` , `U.double` , `U.drag`	the pointer (`U.on` : a mouse moved it and it shows), whether it moved, the wheel's clicks of the frame, a double click, the drag `{b, x0, y0, dx, dy}` (past 3 pixels; `dx, dy` the frame's move)
`U.pressed(b)` , `U.released(b)` , `U.down(b)` , `U.shift()` , `U.drag_arrows(step)`	the button states, the Shift key, and a drag as arrow keys

In the editors: a click chooses a row, a double click (or a click on the chosen one in the dialogs) takes it, the wheel moves or changes a value. In the 3D views the right button dragged turns, the middle one moves, the wheel zooms. On canvases the left button draws and the middle one picks the colour. In bm Code a click

places the cursor, a drag selects, a double click takes the word, a click in the numbers is a breakpoint.

More players

Bluetooth controller *n* is player *n*: pair it from the monitor with `T`, one at a time; each takes the first free place and its light the player's colour (1 blue, 2 red, 3 green, 4 pink).

- The USB keyboard or gamepad and the serial line are the first player without a pad (without Bluetooth pads: player 1).
- The Bluetooth keyboard is a player of its own, the next one without a pad (the first, if there is nothing on USB).
- One-player games use `btn(i)` without `p` and work with any controller. A game for more players asks `btn(i, p)` for each (example: `carts/pong`, two-player mode).
- bmlib has the screen where the players join (`lib.party`), the split screen (`lib.split`) and the colours (`lib.PLAYER_COLORS`); the SDK's *Versus 2D* template uses them.

```
local n = players()                       -- how many controllers there are
if btn(2, 1) then p1.y = p1.y - 3 end     -- up, player 1
if btn(2, 2) then p2.y = p2.y - 3 end     -- up, player 2
local x, y = stick(1)                     -- player 1's stick, from -1 to 1
```

Time and system

Function	What it does
`time()`	seconds since the cartridge started (with decimals)
`stat(n)`	a measurement of the console (table below)
`frameskip([n])`	keeps the game's time at 60 `_update` a second whatever `_draw` costs (see below)
`devkit([mode])`	the dev kit's overlay: `0` off, `1` at the top right, `2` at the bottom right (`3` , the old functions page, is the top: old games keep working); with a mode it shows it (a game's own key for it, for example Select on a pad: F11 is the keyboard's), in this run only: every game starts as Settings says (off by default). Returns the old mode. To go round it: `devkit((devkit() + 1) % 3)`
`profile([on])`	the profiler of functions (see below)
`devinfo(line, ...)`	up to 4 lines of the game in the dev kit's session report ("the game says:", as the run ended: its quality, its actors…), 18 characters each; `devinfo()` none. Call it again when they change
`battery()`	the console's battery: its charge `0` – `100` and `true` on the charger (`local pct, plugged = battery()`); `nil` without a battery (the Pi). On the RGB30 the charge comes from the voltage (an indication: it sags under load and rises on the charger); the charger is seen within a quarter of a second
`battery_low()`	`true` while the battery is low: 20% or less off the charger (until 23%). Then the system draws by itself a small red battery top right over the frame (left of the dev kit's overlay when it is at the top), in every game and tool; *Settings > Screen and sound > Low battery icon* (`battery_icon=0` in `bm/config.txt`) turns that icon off, not `battery_low()` . Information only: each console has its own battery, so in lockstep (`bmnet`) it never changes what the game simulates; use it to show something or to save
`code_tokens(text)`	the **tokens** of a piece of Lua code, counted as `stat(11)` and the SDK's dev kit do (`src/bm/tokens.c`): each name, keyword, number, string and operator is one; comments, spaces, `,` `.` `:` `;` `::` , closing brackets (`)` `]` `}`), `end` and `local` do not count, nor the minus sign in front of a number (`-1` is one token). Information, not a limit: bm puts no ceiling on tokens (nor does the `.b16`)
`log(...)`	writes in the kernel's log (serial line and console), not on the game's screen

Function	What it does
`report(kind, text)`	a report for the people who develop bm: saved in `bm/reports` on the SD card with kernel, branch, board and date (and a first `options` line: the 3D options in force), then sent to the reports' repository if there are `github_token` and the network (`src/kernel/reports.h`); at most 8 per run, 256 KiB each; `true` if saved. `kind` is a required string, a free-text category label: it is turned into lower case with `-` between other characters and goes into the report header and the uploaded file name; with `kind == "session"` the upload replaces the previous file (Overbit uses `"overbit-bench"`). The first time a game calls it, the player is asked (the answer stays in `bm/config.txt` , `allow_...`): after a no, `false`
`shot([prefix])`	saves the frame being drawn as a PNG (see the Screenshots chapter)
`quit()`	closes the cartridge at the end of the frame
`timeslice(co, [k])`	coroutine `co` stops by itself after about `k` thousand Lua instructions in a frame (400 if missing) and `coroutine.resume` returns `true` without values: a long computation goes on in the next frames instead of stopping the cartridge at the instruction limit. `timeslice(nil)` takes it away (nano8 uses it for its cartridges)

Random numbers: `math.random` . For different games at every start, seed the generator when the player presses a key: `math.randomseed(stat(3))` .

`stat(n)`

`n`	Value
0	KiB used by Lua
1	ms of the last frame (`_update + _draw` , with the GPU's 3D)
2	fps
3	frame number
4	3D triangles
5	3D pixels (0 with the GPU)
6	ms spent in 3D drawing (since `zclear` ; with the GPU the ARM's part)
7	3D vertices transformed
8	ms since the start of this frame (to measure the phases)
9	`1` if the GPU draws the 3D

n	Value
10	Lua instructions of the last frame (`_update` + `_draw` , in instructions, rounded to the thousand)
11	dev kit: tokens of the cartridge's code (`code_tokens`)
12	dev kit: the most KiB of Lua of this run
13	dev kit: KiB of the cartridge's data in memory (sprite sheet, map, models and skeletons, sound bank, the 3D z-buffer)
14	dev kit: the Lua instructions of the busiest frame of this run
15	dev kit: how many `_update` ran before this `_draw` (1; more with `frameskip`)
16	dev kit: ms of GPU job time (bin + render) accounted in the last frame, `0` when the ARM draws the 3D (with `stat(1)` it tells whether the GPU or the ARM sets the pace)

BEWARE
`stat` is **a measurement only**: it differs between consoles, so never use it (nor 1, 2, 6, 8, 10) in game logic, or a lockstep match falls out of step.

Heavy frames: `frameskip`

- When a frame takes longer than 1/60 s, up to `n` `_update` run before the next `_draw` (the frames in between are not drawn), so a game that moves 1/60 s per `_update` does not slow down.
- Past `n` the time is let go (the game slows rather than never drawing).
- `1` (the default) is one `_update` a frame; the most is 8. It returns the old value.
- A button pressed counts once in `btnp()` (and `mousep()` , the wheel) however many `_update` see it; `btn()` stays held.
- Overbit uses `frameskip(4)` (its benchmark `1`).

The dev kit and the profiler

The **dev kit** is a performance overlay over any game, **off by default**. Turn it on from Settings > Screen and sound > "Performance overlay" (Off, Top, Bottom: it stays saved), with F11 on the keyboard (a system key, also in the tools), with `p` from the serial line or with `devkit(mode)` from the game: once at the top right, again at the bottom right, again off. F11, `p` and `devkit()` last one run: every game starts (and resumes) as Settings says. It is one small block:

```
60fps 8.2ms           frames a second; ms of work in a frame (the parts below)
cpu [======---]  49%  the CPU's frame in its parts, % of 16.7 ms
gpu [===------]  31%  the GPU's busy time, % of 16.7 ms (only while the GPU draws the 3D)
```

- The CPU bar's parts, each its own colour, left to right: `_update` , `_draw` without its 3D (with the page's clear and copy), the 3D calls (`stat(6)` without the wait for the GPU), the **audio** interrupt, the

wait for the GPU, and the rest free.

- The audio interrupt stops whatever code runs: the console counts its time and takes it out of the part it fell in, so the parts add up.
- The % are of the **16.7 ms of a frame at 60 Hz** (not of the frame: a game at 30 fps shows over 100%), the means of the last second. The % and the bar's frame are green under 50%, yellow up to 80%, red over 80% or when a frame was skipped in that second.
- On the big screens it is bigger (x2 from 1280 wide, x3 at 1920). It costs a few timestamps a frame; off, nothing. The low battery icon stays left of it at the top.

The **profiler of functions**: `profile(true)` measures the functions that cost the most, in ms a frame, the average of the last second (60 frames). It costs nothing when off.

- `profile()` returns up to 32 functions, the costliest first: `{name, where, c, self, total, calls}` , and the frames measured (0 during the first second).
- `where` is `"main.lua:120"` or `"[C]"` ; `c` is true for the console's functions and Lua's written in C; `self` and `total` are ms a frame, its own time and with what it calls; `calls` are the calls a frame, C ones only.
- `profile(true)` turns it on for the run (the session report then lists the ten costliest); `profile(false)` turns it off (and forgets).
- A function with a big `total` and a small `self` spends its time in the others. The console's functions are measured exactly, the Lua ones in pieces of 1000 instructions (enough to see where the time goes).

```
for _, f in ipairs(profile()) do log(f.name, f.self) end
```

The last session's report

When a game ends the dev kit writes one file per game into the reports, `reports/<branch>/session_<game>_<board>.txt` , which the next run of the same game **replaces**. In it: screen, frames and fps, the mean and most time of `_update` + `_draw` , memory, and the time **from one picture to the next** in six classes (up to 18 ms, 25, 34, 50, 100, more); the frame in its parts as the overlay measures them; the audio interrupt's share of the CPU's time; the GPU wait's share and the % of frames where **the GPU set the pace** (the ARM waited more than 2 ms for it); the 12 longest frames and their parts; with the GPU its jobs, the binner's memory and the frame queue; with `profile(true)` the ten costliest functions; and the game's own `devinfo()` lines as the run ended. It goes from the menu when there is a network.

- In `bm/config.txt` : `session_report=0` never, `=1` always; without the line, only with a `github_token` .
- From the code you use `stat(1)` , `stat(2)` , `stat(6)` , `stat(10)` , `stat(11)` – `stat(16)` , `devkit()` and `profile()` . The limit is 20 million instructions per call.
- The **SDK's dev kit** (F1 twice) has the same numbers for the project: tokens, the biggest functions, the data's memory, the file against the 8 MiB of a `.b16` and the numbers of the last try (F5).
- At the end of a run the serial line also writes the line `dev kit: Lua peak ... KiB, data ... KiB, busiest frame ...k instructions, ... frames over 16.7 ms, ... tokens` .

Examples

```
local pct, plugged = battery()
if battery_low() and not warned then save(G) warned = true end   -- save in time
log("frame ms", stat(1))                                         -- on the serial line, not on the scree
```

Saves

Function	What it does
`save(t, [slot])`	saves table `t` on the SD card, in slot 1–8 (default 1); `true`, or `false` and the reason (no SD card, card full…)
`saved([slot])`	the table saved last time in that slot (default 1), or `nil`
`saves()`	the slots in use, `{[slot] = bytes}` (for example `{[1] = 40, [3] = 212}`), and how many there are (8): for a "load game" page
`delsave([slot])`	empties the slot (default 1); `true`, or `false` and the reason (`"nothing saved in slot 3"`)

Good to know

- Each cartridge has **8 save slots**, each a table of up to 32 KiB, in `/bm/save/` on the SD card.
- Slot 1 is `XXXXXXXX.SAV` (the one of always: `save(t)` and `saved()` without a slot), the others `XXXXXXXX.S02` … `.S08`.
- A common use: slot 1 for settings and records (`lib.store` uses it too), the others for the games in progress.
- A table can hold numbers, strings, booleans and other tables (no functions).
- The game's options in the menu (X on the cover) show the bytes of all the slots, and *Delete the save data* deletes them all.

BEWARE
The name of the save files depends on title and author: changing them starts again from zero. Writing on the SD card takes a few milliseconds: call `save()` at moments like the end of a match, not in every frame (`lib.store` and `lib.best` of bmlib write only when a value changes).

A record

Snake's record:

```
function _init()
  local data = saved()
  if data then best = data.best end
end
-- at the end of the match
if score > best then best = score; save({ best = best }) end
```

Several games in progress

Three games in progress, with the page to choose one (slots 2, 3 and 4):

```
local function slots()                  -- the rows of the "load game" page
  local used, rows = saves(), {}
  for i = 1, 3 do
    local p = used[i + 1] and saved(i + 1)
    rows[i] = p and ("Game " .. i .. ": level " .. p.level) or ("Game " .. i .. ": empty")
  end
  return rows
end
function save_game(i) save({ level = level, hp = hp }, i + 1) end
function load_game(i) local p = saved(i + 1) if p then level, hp = p.level, p.hp end end
```

Documents

The player's files, shared by the apps: `/docs` on the SD card (bm Write keeps its `.BMD` there, with the exports `.TXT`, `.MD`, `.HTM` and `.PDF`). An app that makes files for the player (texts, drawings, exports) writes them here.

Function	What it does
`doc_list()`	the files in `/docs`: `{ {name = "LETTER.BMD", size = 1234}, ... }` (empty if there are none); `nil` and the reason if the player did not allow it
`doc_read(name)`	the bytes of a document (a string), or `nil` and the reason
`doc_write(name, bytes)`	writes a document, new or replaced (at most 4 MiB); `true`, or `nil` and the reason
`doc_delete(name)`	deletes a document; `true`, or `nil` and the reason

Good to know

- Names are 8.3 without folders (`"LETTER.BMD"`; lower case is fine, the file is written in upper case). The folder is made by the first write.
- The first time an app uses them the console asks the player, as for the network. The answer stays in `bm/config.txt` (`allow_...=docs=yes`), and after a no every call returns `nil` and the reason.

```
local ok, why = doc_write("NOTES.TXT", "shopping:\nbread\nmilk\n")
if not ok then msg = why end
for _, d in ipairs(doc_list() or {}) do print(d.name .. " " .. d.size) end
```

Screenshots

A game or an app saves the frame it is drawing as a PNG on the SD card, and the player can do the same with a key.

For the player

How	Where
Print Screen on the keyboard	`bm/shots/` on the SD card
Select + L1 + R1 together on a pad (Share + L1 + R1 on a DS4)	`bm/shots/` on the SD card

A box says `Screenshot saved` and the file name for two seconds. The picture is the game's own screen as it was drawn: the dev kit's overlays, the battery icon, the pointer and that box are **not** in it.

> **BEWARE**
> Games and apps run by bm's menu take it; the menu's own screens do not yet.

[TO FILL: check Select + L1 + R1 on the PowKiddy RGB30; not tried on the console yet]

For an app

Function	What it does
`shot([prefix])`	asks for the frame being drawn: it is saved as a PNG in `bm/shots/` when `_draw` ends, and it returns the name it will have (`"bm/shots/PONG0007.png"`). The name starts with the first letters of the game's title, or of `prefix` (up to 4 letters or digits), then a number. `nil` and a message if the numbers are over

Call it at any point of the frame; calling it twice in a frame gives one file. There is no question to the player (the file only goes to `bm/shots/`).

```
local path = shot()          -- "bm/shots/PONG0007.png"
local path = shot("menu")    -- "bm/shots/MENU0001.png"
```

Names and format

- **Name**: `<PREFIX><NNNN>.PNG` , 8.3 as the card's file system wants. The prefix is the first (up to) 4 letters or digits of the game's title (`Pong` → `PONG` , `Hunter's Night` → `HUNT` ; `SHOT` if there are none) or of the `prefix` the app gave.
- **Number**: one more than the highest of that prefix already in `bm/shots/` , from 0001 to 9999. The Pi Zero has no clock, so there is no date.
- **Format**: PNG, 8 bits RGB, the size of the game's screen (640×360, 360×360…). Nothing is lost: the console's colours are 16 bits (RGB565) and are widened to 8 bits. There is no JPG: it spoils pixel art.
- **Size on disk**: a flat screen takes a few KB; noise can take as much as the raw picture (about 0.7 MB at 640×360).

Try it, step by step

1. Start any game from the menu, for example **Pong**.
2. Press **Print Screen** on the keyboard, or hold **Select + L1 + R1** on the pad.
3. Wait for the box that says `Screenshot saved`.
4. Switch the console off and put the SD card in your computer.
5. Open the folder `bm/shots/`: the picture is there, for example `PONG0001.PNG`.

Network (UDP)

For network games. A game is better off with the bmnet library (lobby, sure messages, lockstep), which uses these functions: see the bmnet volume.

- UDP packets up to 1024 bytes; each cartridge has 2 sockets, closed when it ends.
- Addresses are text (`"192.168.1.23"`); `"*"` is the LAN's broadcast.
- The console's network is needed (WiFi or cable): without it `udp_open` returns `nil` .
- In bmhost the same packets go through the PC's sockets (`BMHOST_NET_ID=k` for more consoles on the same PC, `--realtime` to play at 60 frames a second).

Function	What it does
`s, port = udp_open([port])`	a socket on the port (0 or nothing: any); `nil` and the reason if there is no network or free socket. The first time a game opens one, the player is asked whether it may use the network (the answer stays in `bm/config.txt`); after a no, `nil` and the reason
`udp_send(s, address, port, data)`	sends a string (at most 1024 bytes); `true` if it left (UDP: it can be lost)
`data, address, port = udp_recv(s)`	the next packet that arrived, or `nil` ; up to 48 wait in the queue
`udp_close(s)`	closes the socket
`net_ip()`	the console's address, or `nil` without a network
`net_resolve(name)`	the address of a name: `nil` while it looks for it (call it again the frame after), `false` if it does not exist

```
local s = udp_open(0)
if s then
  udp_send(s, "*", 4000, "hello")
  local data, address, port = udp_recv(s)
end
```

[TO FILL: an example of two consoles finding each other with these functions; the sources only describe bmnet for it]

Open points

- **Your first game**: picture to take: shots/api-core-hello.png
- **Drawing**: picture to take: shots/api-core-prompt.png
- **Screenshots**: check Select + L1 + R1 on the PowKiddy RGB30; not tried on the console yet
- **Network (UDP)**: an example of two consoles finding each other with these functions; the sources only describe bmnet for it